

HTML SVG

A Practical, From-Zero Learning Guide

Part 15 — SVG Mastery & Complete Reference

This final part is your SVG reference and revision guide. It brings together the most important syntax, shapes, paths, styling, transforms, animation, accessibility, React patterns, debugging techniques and project ideas.

Goal: Move from learning SVG concepts to confidently designing and building SVG graphics yourself.

1. The SVG Mental Model

SVG is a coordinate-based graphics language. You describe shapes, paths, styles and relationships inside an XML-based document. The browser renders those instructions as scalable graphics.

- SVG has its own coordinate system.
- viewBox controls that coordinate system.
- Elements can be styled with attributes and CSS.
- Transforms change the coordinate system or position.
- defs stores reusable definitions.
- Paths provide the most powerful geometry.
- SVG elements form a DOM tree and can be manipulated with JavaScript.

2. Basic SVG Template

```
<svg
  xmlns="http://www.w3.org/2000/svg"
  viewBox="0 0 200 100"
  role="img"
  aria-labelledby="title"
>
  <title id="title">Example graphic</title>
  <rect x="10" y="10" width="180" height="80" rx="10"/>
</svg>
```

3. Core Elements Cheat Sheet

Element	Purpose
svg	Root SVG container
g	Group elements
rect	Rectangle / rounded rectangle
circle	Circle
ellipse	Ellipse
line	Straight line
polyline	Open connected points
polygon	Closed connected points
path	Lines, curves and arcs
text	Text inside SVG
tspan	Sub-runs / line management inside text
image	Raster or external image
use	Reuse an existing SVG definition
defs	Reusable definitions
symbol	Reusable graphic template
clipPath	Hard clipping boundary
mask	Visibility through masking
linearGradient	Linear color gradient
radialGradient	Radial color gradient
filter	Visual filter effects

4. Coordinate System

The default origin is normally at the top-left of the SVG viewport. x increases to the right and y increases downward.

```
<svg viewBox="0 0 100 100">
  <circle cx="0" cy="0" r="4"/>
  <line x1="0" y1="0" x2="80" y2="0"/>
  <line x1="0" y1="0" x2="0" y2="80"/>
</svg>
```

5. viewBox

The viewBox establishes the internal coordinate system. A common responsive pattern is width="100%" with a stable viewBox.

```
<svg viewBox="0 0 800 400"
  width="100%"
  height="auto">
  ...
</svg>
```

6. Rectangle Reference

```
<rect
  x="10"
  y="20"
  width="120"
  height="60"
  rx="10"
  ry="10"
  fill="none"
  stroke="black"
  stroke-width="2"
/>
```

- x/y = position
- width/height = size
- rx/ry = rounded corners
- fill = interior
- stroke = outline
- stroke-width = outline thickness

7. Circle and Ellipse

```
<circle cx="50" cy="50" r="30"/>

<ellipse
  cx="100"
  cy="50"
  rx="40"
  ry="20"
/>
```

8. Line, Polyline and Polygon

```
<line x1="10" y1="10" x2="80" y2="50"/>

<polyline points="10,80 30,50 50,70 90,20"/>

<polygon points="50,10 90,90 10,90"/>
```

9. Path Command Cheat Sheet

Command	Meaning
M	Move to
L	Line to
H	Horizontal line
V	Vertical line
C	Cubic Bézier curve
S	Smooth cubic Bézier

Q	Quadratic Bézier
T	Smooth quadratic Bézier
A	Elliptical arc
Z	Close path

10. Absolute vs Relative Path Commands

Uppercase commands use absolute coordinates. Lowercase commands use coordinates relative to the current point.

```
M 20 20 L 80 20
m 20 20 l 60 0
```

Relative commands can make manually authored paths easier to move and repeat, while absolute commands can be easier to inspect numerically.

11. Bézier Curves

```
<!-- Cubic -->
<path d="M10 80 C40 10 70 10 100 80"/>

<!-- Quadratic -->
<path d="M10 80 Q55 10 100 80"/>
```

Control points determine the direction and strength of the curve. Think of them as handles that pull the curve.

12. Arc Command

```
<path
  d="M20 50
    A30 30 0 0 1 80 50"
/>
```

The arc command uses radii, rotation, large-arc flag and sweep flag. It is one of the most confusing commands initially, so learn it through small experiments.

13. Fill and Stroke

```
<path
  d="M10 10 L90 10 L50 90 Z"
  fill="gold"
  stroke="black"
  stroke-width="2"
/>
```

- fill paints the interior.
- stroke paints the outline.
- fill="none" removes the fill.
- stroke="none" removes the stroke.

14. Stroke Styling

```
<path
  d="M10 50 L90 50"
  stroke="currentColor"
  stroke-width="8"
  stroke-linecap="round"
  stroke-linejoin="round"
/>
```

- stroke-linecap: butt, round, square
- stroke-linejoin: miter, round, bevel
- stroke-dasharray controls dash patterns
- stroke-dashoffset shifts the dash pattern

15. currentColor

currentColor makes SVG inherit the CSS text color of its context. This is extremely useful for reusable icons.

```

.icon {
  color: tomato;
}

<svg class="icon" viewBox="0 0 24 24">
  <path fill="currentColor" d="..." />
</svg>

```

16. Opacity

```

<circle
  cx="50"
  cy="50"
  r="30"
  fill="blue"
  opacity=".5"
/>

<g opacity=".7">
  ...
</g>

```

17. Gradients

```

<defs>
  <linearGradient id="g"
    x1="0" y1="0"
    x2="1" y2="1">
    <stop offset="0%" />
    <stop offset="100%" />
  </linearGradient>
</defs>

<rect fill="url(#g)"
  width="100"
  height="100"/>

```

18. Patterns

```

<pattern id="grid"
  width="10"
  height="10"
  patternUnits="userSpaceOnUse">
  <path d="M10 0H0V10"
    fill="none"
    stroke="black"/>
</pattern>

<rect width="100" height="100"
  fill="url(#grid)"/>

```

19. Transform Cheat Sheet

```

transform="
  translate(20 10)
  rotate(30)
  scale(1.2)
"

```

- `translate(x y)` moves
- `rotate(angle)` rotates
- `scale(x y)` scales
- `skewX(angle)` skews horizontally
- `skewY(angle)` skews vertically
- Transform order matters.

20. transform-origin

CSS transform-origin can help control where CSS transforms originate. SVG transforms and CSS transforms have historical differences, so test complex cases carefully.

21. Grouping

```

<g fill="none"
  stroke="black"
  stroke-width="2">
  <circle cx="30" cy="30" r="20"/>

```

```

    <line x1="50" y1="30" x2="80" y2="30"/>
  </g>

```

A group lets you apply styles and transforms to several children at once.

22. defs and Reuse

```

<defs>
  <circle id="dot" r="5"/>
</defs>

<use href="#dot"
  x="20" y="20"/>
<use href="#dot"
  x="50" y="40"/>

```

23. clipPath

```

<defs>
  <clipPath id="clip">
    <circle cx="50" cy="50" r="40"/>
  </clipPath>
</defs>

<image
  href="photo.jpg"
  width="100"
  height="100"
  clip-path=url(#clip)"
/>

```

24. mask

Masks control visibility using another graphic's luminance or alpha. They are useful for soft fades and complex reveal effects.

25. Text

```

<text
  x="20"
  y="40"
  font-size="18">
  Hello SVG
</text>

```

Use HTML for normal interface copy and SVG text when the text belongs to the graphic itself.

26. tspan

```

<text x="20" y="30">
  <tspan x="20" dy="0">First line</tspan>
  <tspan x="20" dy="20">Second line</tspan>
</text>

```

27. SVG Animation Basics

```

<circle
  cx="20"
  cy="50"
  r="8">
  <animate
    attributeName="cx"
    from="20"
    to="180"
    dur="2s"
    repeatCount="indefinite"/>
</circle>

```

Modern web projects often prefer CSS animations or JavaScript/React animation libraries, but SVG animation concepts are still useful to understand.

28. CSS Animation

```

.dot {
  animation: move 2s ease-in-out infinite;
}

@keyframes move {
  from { transform: translateX(0); }
  to { transform: translateX(100px); }
}

```

29. Path Drawing Animation

```
.route {
  stroke-dasharray: 500;
  stroke-dashoffset: 500;
  animation: draw 2s ease forwards;
}

@keyframes draw {
  to { stroke-dashoffset: 0; }
}
```

For arbitrary paths, `getTotalLength()` can provide the actual path length instead of guessing.

30. SVG Accessibility Reference

- Decorative SVG → `aria-hidden="true"`.
- Meaningful image → `role="img"` plus accessible name.
- Interactive graphic → keyboard-accessible control and visible focus.
- Complex chart → textual summary or data alternative.
- Do not rely on color alone.
- Respect reduced motion.

31. SVG in React

```
function Star({ size = 24, className = "" }) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      className={className}
      aria-hidden="true"
    >
      <path d="..." fill="currentColor" />
    </svg>
  );
}
```

React converts SVG attributes to JSX-friendly property names in many cases. Examples include `className`, `strokeWidth` and `tabIndex`.

32. React SVG Props

```
<Icon
  size={24}
  className="text-orange-500"
  aria-label="Favorite"
/>
```

A good icon component should expose only the props that consumers genuinely need.

33. React Data Visualization

```
function BarChart({ data }) {
  return (
    <svg viewBox="0 0 500 300">
      {data.map((item, index) => (
        <rect
          key={item.id ?? index}
          x={index * 50}
          y={300 - item.value}
          width="30"
          height={item.value}
        />
      ))}
    </svg>
  );
}
```

Separate data processing from geometry calculations as charts become more complex.

34. Unique SVG IDs in React

```
const id = useId();
```

```
const gradientId = `gradient-${id}`;  
  
<linearGradient id={gradientId}>  
  ...  
</linearGradient>  
  
<rect fill={`url(#${gradientId})`} />
```

35. SVG Events

```
<circle  
  onClick={handleClick}  
  onMouseEnter={handleEnter}  
  onMouseLeave={handleLeave}  
>  
  />
```

For keyboard accessibility, prefer wrapping the visual in a native button when appropriate.

36. SVG Pointer Interaction

pointer events can be useful when building interactive maps and diagrams. Decide whether an element should receive pointer events or allow events to pass through.

37. SVG Debugging Checklist

- Check the viewBox.
- Check x/y values.
- Check width/height.
- Temporarily remove transforms.
- Temporarily remove filters.
- Inspect fill/stroke.
- Check clipping and masks.
- Look for duplicate IDs.
- Check element order.
- Inspect the browser console.

38. Common SVG Errors

Problem	Likely Cause
Nothing visible	Wrong coordinates, fill/stroke or viewBox
Gradient missing	Wrong or duplicate ID
Shape clipped	clipPath, mask or viewBox
Unexpected position	Transform order or nested transforms
Text missing	Font, color or coordinates
SVG stretches	Missing/incorrect viewBox or aspect handling
Animation wrong	CSS/SVG transform origin or timing
React warning	Invalid JSX SVG attribute

39. SVG Learning Order

- 1. Basic SVG structure
- 2. Coordinate system and viewBox
- 3. Shapes
- 4. fill/stroke
- 5. paths

- 6. curves and arcs
- 7. transforms
- 8. groups and defs
- 9. gradients and patterns
- 10. clipPath and masks
- 11. text
- 12. animation
- 13. accessibility
- 14. React integration
- 15. optimization and production

40. Practice Exercise Ladder

- Draw a traffic light.
- Draw a house using basic shapes.
- Create a simple logo.
- Recreate a food icon.
- Make a curved route.
- Create a progress ring.
- Build a bar chart.
- Build a line chart.
- Create an interactive map marker.
- Animate a delivery route.
- Build a complete SVG dashboard.

41. Mini Project — Food Delivery Pin

```
<svg viewBox="0 0 100 120">
  <path
    d="M50 115
      C50 115 15 78 15 48
      A35 35 0 1 1 85 48
      C85 78 50 115 50 115Z"
    fill="currentColor"/>
  <circle
    cx="50" cy="48" r="12"
    fill="white"/>
</svg>
```

Improve it with a restaurant icon, hover state and accessible label.

42. Mini Project — Progress Ring

```
<svg viewBox="0 0 120 120">
  <circle
    cx="60" cy="60" r="45"
    fill="none"
    stroke="lightgray"
    stroke-width="10"/>

  <circle
    cx="60" cy="60" r="45"
    fill="none"
    stroke="currentColor"
    stroke-width="10"
    stroke-linecap="round"
    transform="rotate(-90 60 60)"
    stroke-dasharray="282.7"
    stroke-dashoffset="70.7"/>
</svg>
```

The circumference is approximately $2\pi r$. That relationship is the foundation of many SVG progress rings.

43. Mini Project — Order Status Timeline

- Create circles for each status.
- Connect them with lines.
- Use `currentColor` for the active state.
- Add labels outside the SVG if possible.
- Make the active status available to screen readers.
- Animate the progress only when motion is appropriate.

44. Mini Project — Restaurant Analytics

- Orders line chart.
- Food category donut chart.
- Average preparation time bar chart.
- Delivery progress ring.
- Responsive dashboard.
- Accessible data summaries.

45. SVG Design Principles

- Start with the simplest geometry.
- Use a clear coordinate system.
- Reuse definitions.
- Keep styles consistent.
- Do not overuse effects.
- Design for multiple sizes.
- Treat accessibility as part of design.
- Measure performance when graphics become complex.

46. SVG vs Image Formats

- SVG → logos, icons, diagrams, illustrations, charts.
- PNG → transparency and lossless raster graphics.
- JPEG → photographic content.
- WebP/AVIF → modern compressed raster content.
- Choose the format based on the content rather than forcing every asset into SVG.

47. Production Decision Tree

- Need scalable geometry? → SVG.
- Need individual interactive shapes? → SVG.
- Need thousands/millions of rapidly changing particles? → consider Canvas/WebGL.
- Need normal page layout? → HTML/CSS.
- Need a photograph? → raster image format.

48. Final CraveCart SVG Challenge

Build an SVG-powered admin analytics dashboard for CraveCart.

- 1. Order trend line chart.
- 2. Category donut chart.
- 3. Delivery progress ring.
- 4. Restaurant status indicators.
- 5. Animated delivery route.
- 6. Responsive viewBox.
- 7. Reusable React components.
- 8. Unique IDs with useId.
- 9. Accessible chart summaries.
- 10. Loading/error/empty states.
- 11. Reduced-motion support.
- 12. Optimized SVG assets.

49. Suggested Component Structure

```
AdminDashboard/  
  charts/  
    OrdersChart.jsx  
    CategoryDonut.jsx  
    DeliveryRing.jsx  
    RestaurantActivity.jsx  
  icons/  
    RestaurantIcon.jsx  
    DeliveryIcon.jsx  
  shared/  
    ChartEmpty.jsx  
    ChartError.jsx  
    ChartSkeleton.jsx
```

50. Final SVG Master Checklist

- I understand viewBox.
- I can draw basic shapes.
- I understand fill and stroke.
- I can write simple paths.
- I understand M/L/H/V/C/Q/A/Z.
- I can use transforms.
- I can group elements.
- I can use defs and use.
- I can create gradients.
- I understand clipPath and masks.
- I can animate SVG.
- I can make SVG responsive.
- I know how to make SVG accessible.
- I can use SVG in React.
- I understand unique IDs.
- I can debug SVG with DevTools.
- I know when SVG is not the right tool.
- I can optimize SVG for production.

51. Final Revision Cheat Sheet

```
<svg viewBox="0 0 100 100">...</svg>

<rect x="10" y="10" width="20" height="20"/>
<circle cx="50" cy="50" r="20"/>
<line x1="0" y1="0" x2="100" y2="100"/>
<polyline points="0,0 20,30 50,10"/>
<polygon points="50,0 100,100 0,100"/>
<path d="M10 10 L90 10 C... Z"/>

fill="currentColor"
stroke="currentColor"
stroke-width="2"

transform="translate(...) rotate(...) scale(...)"

<defs>...</defs>
<use href="#id"/>

viewBox="minX minY width height"
preserveAspectRatio="xMidYMid meet"
```

52. What to Do Next

Do not stop at reading. The fastest way to master SVG is to build small graphics repeatedly. Recreate real icons, diagrams and dashboard components from memory, then compare your solution with the reference.

For your CraveCart project, SVG is especially useful for admin analytics, delivery routes, progress indicators, category charts, custom icons, empty states and interactive maps.

53. Course Complete ■

You have now covered the SVG journey from basic HTML embedding through shapes, paths, curves, transforms, styling, gradients, clipping, masking, animation, accessibility, React, optimization, security and production architecture.

Your next step is practice: choose one visual from CraveCart and build it entirely with SVG. Start simple, then improve it using the techniques from all 15 parts.